UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Estudo Comparativo de Variantes de *Autoencoders* no Algoritmo de Agrupamento Profundo *k-Deep Autoencoder*

Magnus Brigido Paulo Freire

Orientador: Prof. Dr. Leonardo Enzo Brito da Silva

Natal, RN, 15 de dezembro de 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Estudo Comparativo de Variantes de *Autoencoders* no Algoritmo de Agrupamento Profundo *k-Deep Autoencoder*

Magnus Brigido Paulo Freire

Orientador: Prof. Dr. Leonardo Enzo Brito da Silva

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 15 de dezembro de 2025

Divisão de Serviços Técnicos

Catalogação da publicação na fonte. UFRN / Biblioteca Central Zila Mamede

Freire, Magnus Brígido Paulo.
Estudo Comparativo de Variantes de *Autoencoders* no Algoritmo de Agrupamento Profundo *k-Deep Autoencoder* / Magnus Brígido Paulo Freire. - 2025.
37 f.: il.

Monografia (graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia de Computação, Natal, RN, 2025.
Orientação: Prof. Dr. Leonardo Enzo Brito da Silva.

1. Inteligência Artificial - Monografia. 2. Aprendizado de Máquina - Monografia. 3. Autoencoder - Monografia. I. Silva, Leonardo Enzo Brito da. II. Título.
RN/UF/BCZM CDU 004.8

Elaborado por Fernanda de Medeiros Ferreira Aquino - CRB-15/301

Estudo Comparativo de Variantes de *Autoencoders* no Algoritmo de Agrupamento Profundo *k-Deep Autoencoder*

Magnus Brigido Paulo Freire

Monografia aprovada em 15 de dezembro de 2025, pela banca examinadora composta pelos seguintes membros:

Prof. Dr. Leonardo Enzo Brito da Silva (orientador) . . . . . IMD/UFRN

Prof. Dr. Daniel Lopes Martins . . . . . IMD/UFRN

Prof. Dr. Silvan Ferreira da Silva Junior . . . . . IMD/UFRN

Agradecimentos

Agradeço à mim por ter persistido após tantos percalços.

À minha família pelo apoio durante toda a jornada.

Ao meu orientador, sou grato pela orientação, paciência e disposição.

À banca pelas orientações e sugestões para melhoria final deste trabalho.

Resumo

Este projeto investiga o impacto de diferentes variantes de autoencoders no desempenho do algoritmo de agrupamento profundo k-Deep Autoencoder (k-DAE), baseado na reconstrução de cada amostra por autoencoders especialistas. Foram avaliados os modelos de autoencoder padrão (AE), Denoising (DAE) e Esparso (SAE) utilizando a base de dígitos manuscritos MNIST, seguindo a metodologia de pré-treinamento e treinamento descritas no artigo k-Deep Autoencoder. A qualidade dos agrupamentos foi medida pelo Adjusted Rand Index (ARI) ao variar a dimensionalidade do espaço latente. Os resultados indicam que o DAE apresenta maior robustez, em relação à capacidade de generalização, e estabilidade em diferentes dimensionalidades, enquanto o AE e SAE demonstraram sensibilidade a essa variação. A análise dos hiperparâmetros definidos, o dropout para o modelo Denoising e a taxa de regularização para o modelo Esparso, evidenciou que os valores adotados (50% de dropout e 10^{-4} de penalidade) não foram valores em que os modelos apresentaram os piores desempenhos. Produzindo uma comparação balanceada entre os modelos e garantindo a consistência da avaliação.

Palavras-chave: Inteligência Artificial, Aprendizado de Máquina Não Supervisionado, Algoritmos de Agrupamento, Autoencoder.

Abstract

This project investigates the impact of different autoencoder variants on the performance of the deep clustering algorithm k-Deep Autoencoder (k-DAE), which is based on the reconstruction of each sample by specialized autoencoders. The study evaluated the standard autoencoder (AE), Denoising Autoencoder (DAE), and Sparse Autoencoder (SAE) models using the MNIST handwritten digits dataset, following the pre-training and training methodology described in the k-Deep Autoencoder paper. Clustering quality was measured using the Adjusted Rand Index (ARI) while varying the dimensionality of the latent space. The results indicate that the DAE exhibits greater robustness and stability across different latent dimensions, whereas the AE and SAE models showed sensitivity to these variations. The analysis of the defined hyperparameters (dropout for the Denoising model and regularization rate for the Sparse model) showed that the adopted values (50% dropout and 10^{-4} penalty) were not those at which the models exhibited their worst performance. This ensures a balanced comparison among the models and preserves the consistency of the evaluation..

Keywords: Artificial Intelligence, Unsupervised Machine Learning, Clustering Algorithms, Autoencoder

Sumário

1 INTRODUÇÃO 13

2 FUNDAMENTAÇÃO TEÓRICA 17
2.1 Agrupamento 17
2.1.1 K-means 17
2.2 Métrica de Avaliação 18
2.3 Redes Neurais 19
2.3.1 Autoencoders 20
2.3.1.1 Autoencoder Clássico 20
2.3.1.2 Denoising Autoencoder 20
2.3.1.3 Sparse Autoencoder 21
2.3.2 Algoritmo *k*-Deep-AutoEncoder (*k*-DAE) 21

3 METODOLOGIA 25
3.1 Base de dados 27
3.1.1 Experimento 1 27
3.1.2 Experimento 2 28
3.1.3 Experimento 3 28

4 RESULTADOS E DISCUSSÕES 29
4.1 Resultados 29
4.1.1 Experimento 1 29
4.1.2 Experimento 2 29
4.1.3 Experimento 3 30

5 CONCLUSÃO 35

REFERÊNCIAS 37

Capítulo 1

Introdução

Dentro da computação, os problemas são resolvidos por meio da implementação de algoritmos, que especificam passo a passo como determinada tarefa deve ser executada. No entanto, desenvolver tais algoritmos nem sempre é trivial, principalmente em problemas que envolvem reconhecimento de padrões. Tarefas que são simples para um ser humano podem apresentar complexidade elevada para serem executadas por programas de computador. Além disso, a quantidade de tarefas complexas que precisam ser realizadas diariamente é muito grande, tornando inviável que seres humanos sejam capazes de implementar algoritmos capazes de realizar essas tarefas de maneira eficiente. Dessa forma, tornou-se essencial capacitar máquinas para que sejam capazes de realizem tarefas que normalmente exigiriam intervenção humana (FACELI et al., 2011).

Para resolver esse desafio, surgiram técnicas de Inteligência Artificial (IA), especialmente aquelas pertencentes à subárea da IA, o Aprendizado de Máquina (AM), que demonstraram eficiência na execução de tarefas complexas de forma autônoma. Inicialmente, muitas dessas técnicas dependiam do conhecimento de especialistas, que precisava ser codificado manualmente em programas de computador. Esse processo apresentava limitações significativas: dependência de profissionais especializados, subjetividade na codificação do conhecimento e dificuldade de tradução para termos que uma máquina pudesse interpretar. Com o avanço das pesquisas, métodos que permitem às máquinas aprender diretamente a partir dos dados disponíveis passaram a ser desenvolvidos, tornando as máquinas capazes de adquirir conhecimento e melhorar seu desempenho sem a necessidade de uma intervenção humana.

O Aprendizado de Máquina (AM) (MURPHY, 2022; PRINCE, 2023; LUGER, 2013) é a subárea da Inteligência Artificial dedicada ao desenvolvimento de modelos matemáticos capazes de identificar padrões em dados, tomar decisões e aprimorar seu desempenho por meio da experiência. Nessa abordagem, os parâmetros dos modelos são ajustados automaticamente com base nas informações fornecidas, permitindo que o sistema aprenda a produzir previsões mais precisas e representações mais consistentes dos dados. A quantidade de dados disponíveis é fundamental, pois quanto maior o seu volume, maior a oportunidade do modelo aprender a partir da experiência. Existem três tipos principais de aprendizado de máquinas: Aprendizado por Reforço, Aprendizado Supervisionado e Aprendizado Não Supervisionado.

No Aprendizado por Reforço (MURPHY, 2022; PRINCE, 2023; LUGER, 2013) um

agente (uma entidade que percebe e toma decisões em ambiente) interage com um ambiente (podendo ser definido como um cenário onde o agente atua), se comportando de forma determinística ou estocástica, com o objetivo de maximizar a recompensa (atribui a qualidade da ação executada) acumulada ao longo do tempo (retorno). Essa interação é feita através de ações tomadas em cada estado (que representa uma localização que compõe um ambiente), resultando em uma transição para um estado subsequente e em um ganho de recompensa, que pode ser positiva ou negativa.

O Aprendizado Supervisionado (MURPHY, 2022; PRINCE, 2023; LUGER, 2013) baseia-se no mapeamento dos dados de entrada para os dados de saída. Os dados compõem as características de uma amostra, cujo padrão está relacionado a um rótulo previamente conhecido. Durante o treinamento, o modelo ajusta seus parâmetros minimizando a função de perda, aproximando as suas previsões das respostas reais. Após o treinamento, espera-se que o modelo seja capaz de produzir previsões adequadas sobre quaisquer dados novos cujos rótulos são desconhecidos. As principais tarefas do Aprendizado Supervisionado são classificação e regressão. Na tarefa de classificação o objetivo é a atribuição de um rótulo discreto a uma amostra. Por exemplo, identificar se um paciente possui ou não uma determinada doença representa uma classificação binária, enquanto a identificação de uma doença dentre várias opções constitui uma classificação multiclasses. Já em uma tarefa de regressão, os rótulos são valores contínuos que mapeiam os dados para dar respostas preditivas, como estimar o valor de uma casa a partir das características que possui, ou o ponto de congelamento e ebulição dada uma estrutura química.

O Aprendizado Não Supervisionado (MURPHY, 2022; PRINCE, 2023; LUGER, 2013), ao contrário do Aprendizado Supervisionado, possui como principal característica a ausência de rótulos no conjunto de treinamento. Com isso, o processo de aprendizado não é baseado no mapeamento da entrada para uma saída, mas sim na identificação da estrutura e dos padrões presentes nos dados. As principais tarefas do Aprendizado Não Supervisionado são redução de dimensionalidade e agrupamento. A Redução de Dimensionalidade visa diminuir a complexidade dos dados, preservando suas características essenciais. Isso faz com que seja produzido um espaço de dimensão reduzida, espaço latente, onde os dados possuem as informações mais significativas do conjunto original. O Agrupamento é uma tarefa muito comum neste paradigma, e é um dos temas principais deste trabalho.

O algoritmo k-DAE, apresentado por Opochinsky (OPOCHINSKY et al., 2020) é mostrado como uma extensão e variante do algoritmo k-means. Também sendo definido como uma alternativa mais simples, em termos de arquitetura, aos modelos anteriores, como o Deep Autoencoder Mixture Clustering (DAMIC), apresentado no artigo (CHAZAN; GANNOT; GOLDBERGER, 2019).

O objetivo deste trabalho é estudar, implementar e avaliar o desempenho do algoritmo k-Deep Autoencoder (k-DAE) na tarefa de agrupamento. Para isso, diferentes variantes de autoencoders serão utilizadas: o modelo clássico (base), o Denoising Autoencoder e o Sparse Autoencoder. A análise proposta visa compreender o impacto do algoritmo no desempenho do agrupamento, bem como investigar como cada modelo influencia a qualidade das representações aprendidas e dos agrupamentos gerados. Outras análises que serão feitas são: o impacto da variação do Dropout no Denoising e o impacto da

variação do termo de regularização no Sparse.

As seções seguintes apresentarão, nesta ordem, a fundamentação teórica, a metologia utilizada para produção dos experimentos, os resultados produzidos em cada experimento e as discussões sobre eles, e, por fim, a conclusão deste trabalho.

Capítulo 2

Fundamentação Teórica

2.1 Agrupamento

O objetivo de agrupar dados é particionar os dados de entrada em regiões que contenham pontos similares. Essa similaridade é dada pela proximidade dos pontos em um espaço, que pode ser dada pela distância Euclidiana.

2.1.1 K-means

Um algoritmo muito conhecido e utilizado é o K-means, que corresponde a um limite não-probabilístico particular de misturas Gaussianas (BISHOP; BISHOP, 2024). O algoritmo se baseia no cálculo de distância, como a Euclidiana, para definição de grupos. Uma inicialização comum do algoritmo consiste na escolha de K pontos aleatórios que servirão de centróides (μ_k) iniciais, representando cada um um grupo distinto. Após a inicialização, cada ponto é atribuído ao grupo do centróide mais próximo, com base na equação:

$$z_n^* = \arg\min_k \|x_n - \mu_k\|_2^2 \tag{1}$$

onde z_n^* é a atribuição ótima de um grupo para a amostra x_n, e $\arg\min_k$ indica o grupo k que minimiza a distância euclidiana, ou a norma L2.

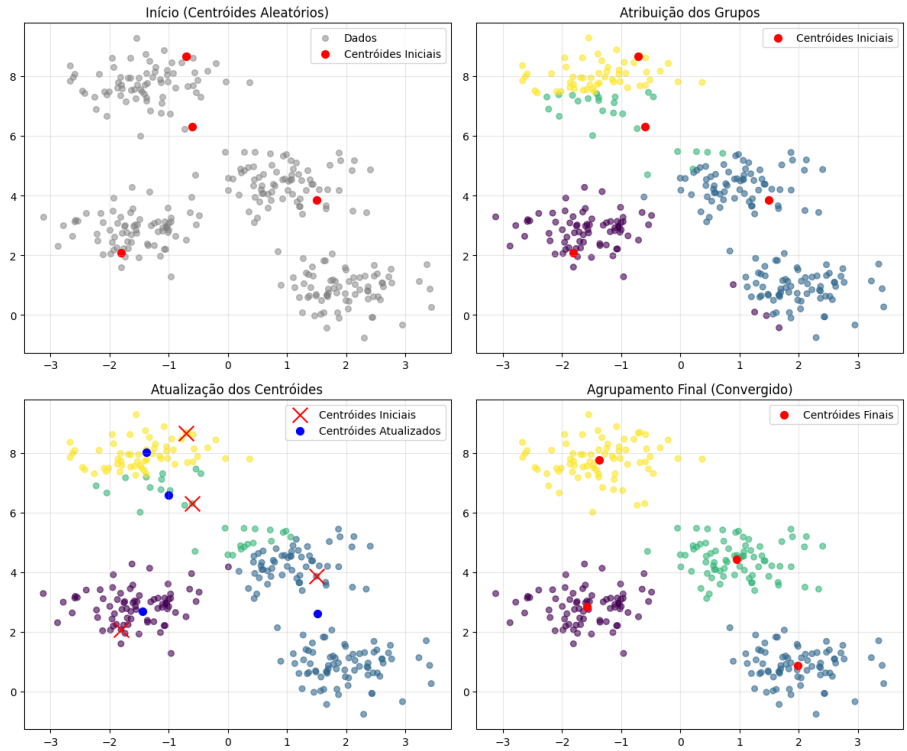
Após a atribuição de cada ponto a um grupo, os centróides são atualizados com a média dos pontos que compõem o seu grupo:

$$\mu_k = \frac{1}{N_k} \sum_{n:z_n=k} x_n \tag{2}$$

onde N_k representa o número de amostras pertencentes ao grupo k e $n : z_n = k$ são os índices dessas amostras.

Os passos de atribuição e de atualização são repetidos até a convergência, que ocorre quando não há mais alterações nas atribuições ou posicionamento dos centróides (MURPHY, 2022; BISHOP; BISHOP, 2024; MAINI; SABRI, 2017), como mostrado a figura 1.

Figura 1 – Processo de agrupamento do algoritmo K-means.



Fonte: autor.

2.2 Métrica de Avaliação

Devido à ausência de rótulos, a avaliação da qualidade das soluções encontradas por algoritmos de agrupamento pode ser efetuada utilizando índices de validação (MURPHY, 2022; XU; Wunsch II, 2009). Nesse sentido, uma das métricas mais utilizadas é o Rand Index (RI), dado por:

$$RI \triangleq \frac{A+B}{A+B+C+D} \tag{3}$$

onde A (verdadeiros positivos) é o número de pares de amostras que estão no mesmo grupo tanto na partição estimada quanto na de referência, B (verdadeiros negativos) representa o número de pares de amostras que estão em grupos diferentes em ambas as partições, C (falsos positivos) corresponde aos pares de amostras que estão no mesmo grupo na partição estimada, mas em grupos diferentes na referência, e D (falsos negativos) representa os pares de amostras que estão em grupos diferentes na partição estimada, mas no mesmo grupo na referência. Sendo produzidos valores entre 0 e 1, em que 1 representa concordância total e 0 o oposto.

Para evitar valores altos de RI mesmo em agrupamentos aleatórios, utiliza-se um ajuste. Logo:

$$ARI \triangleq \frac{\text{RI} - \text{RI esperado}}{\text{RI máximo} - \text{RI esperado}} \tag{4}$$

Dessa forma, o intervalo fica entre -1 e 1, onde 1 representa concordância perfeita, 0 considera o desempenho equivalente ao de um agrupamento aleatório e -1 representa discordância máxima.

2.3 Redes Neurais

A partir de estudos de processamento de informações que ocorrem no cérebro humano surgiu o modelo de Neurônio Artificial, que deu origem às Redes Neurais Artificiais (RNA). Essas redes apresentam grande flexibilidade e podem ser aplicadas em todos os paradigmas de aprendizado de máquina. Baseando-se nas sinapses, foram desenvolvidos modelos matemáticos simples para reproduzir esse comportamento. O funcionamento de um neurônio pode ser expresso pelas equações:

$$a = \sum_{i=1}^{M} w_i x_i \tag{5}$$

$$y = f(a) \tag{6}$$

Sendo $x_1, ..., x_M$ a representação das M entradas correspondentes às ativações dos neurônios da camada anterior e $w_i, ..., w_M$ são variáveis contínuas que representam a força das conexões associadas, denominadas de peso, que assumem valores positivos (excitadores) ou negativos (inibidores), dependendo do comportamento da conexão (FACELI et al., 2011). A pré-ativação a passa pela função de ativação $f(\cdot)$, produzindo uma saída y, chamada de ativação (BISHOP; BISHOP, 2024). A função de ativação é fundamental para introduzir não linearidade e aprender representações complexas.

A arquitetura de uma RNA define a quantidade de camadas, quantidade de neurônios que cada camada possui e como se conectam. Por sua vez, o aprendizado ajusta os pesos das conexões de acordo com os dados (FACELI et al., 2011). A arquitetura mais comum de RNA é composta por, no mínimo, 3 camadas, camada de entrada, camada oculta (intermediária) e camada de saída, e utilizam algoritmos de otimização para o treinamento, que minimizam uma função de perda, a qual mede a diferença entre a saída da rede e os valores esperados.

O algoritmo mais utilizado é o de Retropropagação (Backpropagation), que é estruturado em duas fases: forward e backward. A primeira fase é a de propagação das informações da camada de entrada até a de saída, percorrendo toda a rede. O valor de saída é então comparado ao valor real e a diferença indica o erro da rede. Na segunda fase, esse erro é retropropagado pelas camadas, ajustando os pesos da rede utilizando os gradientes da função de perda em relação aos próprios pesos da rede. O erro de cada neurônio de uma camada oculta é estimado pela soma dos erros dos neurônios ligados a ele, ponderada pelos pesos associados às ligações (FACELI et al., 2011).

Durante o treinamento, as redes neurais podem sofrer sobreajuste (overfitting), que é quando a rede perde a capacidade de generalização. Para mitigar isso, são aplicadas

técnicas como normalização, regularização Dropout (desligamento aleatório de neurônios na camada de entrada da rede) e a Parada Antecipada (early stopping), que monitora a taxa de erro e encerra o treinamento quando a taxa para de decrescer.

A característica que define uma rede neural como profunda é a quantidade de camadas ocultas que ela possui entre a camada de entrada e a de saída. A presença dessas camadas adicionais permite que o modelo construa representações progressivamente mais abstratas à medida que os dados avançam pela rede e passam por sucessivas transformações não lineares. Entretanto, as redes profundas apresentam desafios adicionais, como a instabilidade dos gradientes durante o processo de backpropagation, que podem se tornar muito pequenos ou excessivamente grandes. Além das técnicas citadas para evitar o overfitting, são utilizados algoritmos de otimização, como o Adam, para aumentar a estabilidade e a eficiência do treinamento (BISHOP; BISHOP, 2024).

2.3.1 Autoencoders

Um importante objetivo do aprendizado profundo é descobrir representações dos dados que sejam capazes de solucionar diferentes aplicações. O Autoencoder é uma rede neural auto-associativa bem estabelecida de aprendizado de representações internas. Ela é composta por duas partes: a função codificadora (encoder) e a função decodificadora (decoder), ambas possuindo o mesmo número de neurônios. Ao passar pelo encoder, os dados de entrada são mapeados para uma representação latente $z(x)$, e, passando pelo decoder, a representação oculta é mapeada para saída. É esperado que a diferença, normalmente medida por uma função de perda como o erro médio quadrático, entre os dados de entrada e os dados reconstruídos seja mínima.

2.3.1.1 Autoencoder Clássico

O autoencoder clássico impõe a limitação de dimensionalidade $L < D$ (sendo L a dimensão do espaço latente e D a de entrada) formando um gargalo (bottleneck) que é necessário para que a rede aprenda soluções não triviais (BISHOP; BISHOP, 2024).

A função de perda pode ser escrita como:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N} \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{x}_n\|^2 \tag{7}$$

onde $\mathbf{y}(\mathbf{x}_n, \mathbf{w})$ é a saída do decoder para a entrada $\mathbf{x}_n$ e $\mathbf{w}$ os parâmetros da rede.

2.3.1.2 Denoising Autoencoder

O Denoising Autoencoder (DAE) treina sua rede para reconstruir a partir de dados corrompidos $\tilde{x_n}$. A variável de entrada pode ser corrompida aplicando a técnica de Dropout ou adicionando ruído Gaussiano na camada de entrada (BISHOP; BISHOP, 2024; GéRON, 2023). a técnica Dropout, que será utilizada nesse trabalho para adicionar ruído, é uma forma de regularização muito utilizada e efetiva, que consiste no desligamento de neurônios da camada de entrada e suas conexões aleatoriamente, o que previne sobreajuste e pode ser visto como uma forma implícita de média aproximada de múltiplos

modelos, sem a necessidade de treinar redes separadas (BISHOP; BISHOP, 2024). A saída ideal para este modelo de autoencoder é a própria entrada sem o ruído. Durante o treinamento, a rede é otimizada para reconstruir os dados originais (não corrompidos) a partir de entradas corrompidas, minimizando a função de perda na forma:

$$E(\mathbf{w}) = \sum_{n=1}^{N} \|\mathbf{y}(\tilde{\mathbf{x}}_n, \mathbf{w}) - \mathbf{x}_n\|^2 \tag{8}$$

sendo $\mathbf{y}(\tilde{\mathbf{x}}_n, \mathbf{w})$ a saída do decoder para a entrada corrompida $\tilde{\mathbf{x}}_n$.

2.3.1.3 Sparse Autoencoder

O Sparse Autoencoder (SAE) impõe um tipo diferente de restrição sobre o espaço latente, incentivando que apenas uma parte das ativações seja significativamente diferente de zero. Essa característica promove representações esparsas, o que reduz a dimensionalidade efetiva da região latente (BISHOP; BISHOP, 2024).

Uma forma de introduzir esparsidade é a adição de um termo de regularização L_1 (Lasso), sobre as ativações de uma das camadas ocultas de forma que a nova função de perda regularizada é dada por:

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda \sum_{k=1}^{K} |z_k| \tag{9}$$

onde $E(\mathbf{w})$ é o erro dado pela equação 7 (BISHOP; BISHOP, 2024). O parâmetro λ controla o nível de esparsidade imposto ao modelo, reforçando a penalização quando possui valores maiores e tornando as representações mais esparsas.

2.3.2 Algoritmo *k*-Deep-AutoEncoder (*k*-DAE)

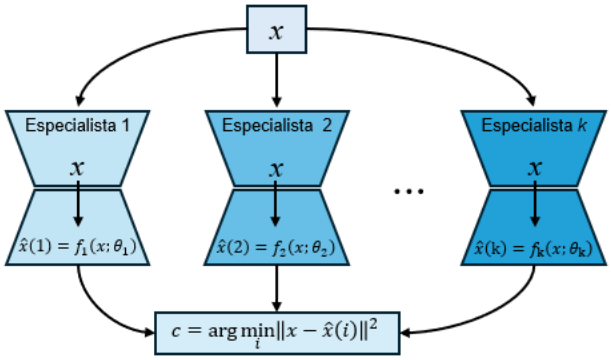
O algoritmo de agrupamento profundo k-DAE, mostrado no artigo (OPOCHINSKY et al., 2020), utiliza-se do conceito do k-means e apresenta a abordagem através da substituição do uso de centróides por autoencoders especializados. A ideia central é que, em um grupo formado por dados semelhantes, o autoencoder especializado deve ter o menor erro de reconstrução dentre os demais autoencoders que estão especializados em outros grupos.

O funcionamento do algoritmo se dá pela atribuição dos dados ao grupo do autoencoder que os melhor reconstruiu e pela atualização dos k autoencoders para melhor reconstruir seus respectivos grupos. Dessa forma, o método realiza simultaneamente o agrupamento e o aprendizado das representações que caracterizam cada grupo.

Um autoencoder é representado por $f_i(x_t; \theta_i)$ e associado a um grupo i, onde θ_i é o conjunto de parâmetros da rede do autoencoder. O objeto reconstruído $f_i(x_t; \theta_i) \in \mathbb{R}^d$ pode ser visto como um centróide orientado a dados do grupo i (OPOCHINSKY et al., 2020).

O agrupamento é realizado minimizando a função de perda de reconstrução, que representa a soma dos menores erros de reconstrução:

Figura 2 – Diagrama de bloco do k-DAE.



Fonte: adaptação feita pelo autor a partir do diagrama apresentado em (OPOCHINSKY et al., 2020).

$$L(\theta_1, ..., \theta_k) = \sum_{t=1}^{n} \min_{i=1}^{k} d(x_t, \hat{x}_t(i)) \tag{10}$$

onde $\hat{x}_t(i) = f_i(x_t; \theta_i)$ é a reconstrução de x_t pelo i-ésimo autoencoder. Sendo d a distância euclidiana quadrática definida por:

$$d(x_t, \hat{x}_t(i)) = \|x_t - \hat{x}_t(i)\|^2.$$

O gradiente é:

$$\frac{\partial L}{\partial \theta_i} = \sum_{t \in N_i} (x_t - \hat{x}_t(i))^\top \frac{d}{d\theta_i} f_i(x_t; \theta_i) \tag{11}$$

A atribuição de cada ponto x_t ao autoencoder que o melhor reconstruiu é feita de forma rígida. Considerando N_i como o conjunto de todos os dados que são melhores reconstruídos pelo i-ésimo autoencoder, temos:

$$N_i = \{t | i = \arg\min_j \|x_t - \hat{x}_t(j)\|\}.$$

Ao contrário de outros agrupamentos profundos propostos, não há risco de colapso para uma solução trivial, onde todos os pontos são mapeados para o mesmo grupo. Pois o objetivo é minimizar o erro de reconstrução, logo é naturalmente melhor utilizar k diferentes autoencoder para reconstrução. Portanto, não há necessidade de adicionar termos de regularização à função de perda para prevenir o colapso dos dados (OPOCHINSKY et al., 2020).

A relação tão próxima com o k-means resultou em o k-DAE ser nomeado dessa forma. Pois ao substituir um autoencoder por uma função constante $f_i(x_t; \theta_i) = \mu_i \in \mathbb{R}^d$, obtém-se o k-means clássico. Assim, a redução da função de perda ao k-means é:

$$L(\mu_1, ..., \mu_k) = \sum_{t=1}^{n} \min_{i=1}^{k} \|x_t - \mu_i\|^2 \tag{12}$$

No entanto, devido a dificuldade de otimização de tarefas de agrupamento com aprendizado não supervisionado, é crucial a inicialização dos parâmetros da rede para o k-DAE (OPOCHINSKY et al., 2020), que é feita através do pré-treinamento do autoencoder e os autoencoders especialistas, como será abordado no capítulo 3.

Capítulo 3

Metodologia

A metodologia seguida para realização dos experimentos do projeto foi a descrição do processo de implementação informada no artigo (OPOCHINSKY et al., 2020), cuja arquitetura da rede e o procedimento final de agrupamento é mostrada na figura 2. Essa decisão foi tomada buscando permanecer mais fiel ao modo em que o algoritmo k-Deep Autoencoder foi apresentado.

Como a inicialização dos parâmetros da rede ainda é crucial, a implementação inicia com o pré-treinamento de um único autoencoder, utilizando todo o conjunto de dados. Então, o k-means é aplicado no espaço latente z gerado pelo autoencoder pré-treinado. Os rótulos gerados pelo k-means inicializam os valores de agrupamento. Após isso, os pontos atribuídos ao i-ésimo grupo são utilizados para pré-treinar o i-ésimo autoencoder $f_i(x;\theta_i)$. Após toda a etapa de pré-treinamento especializado, os autoencoders são treinados conjuntamente para minimizar a reconstrução definida pela função de perda (10). O pseudocódigo está apresentado na figura 3.

Os experimentos foram feitos utilizando computador pessoal e os ambientes de execução Google Colab e o Kaggle. O uso de locais variados para executar os experimentos foram escolhidos para acelerar e observar o intervalo de resultados produzidos, averiguando instabilidades ou resultados destoantes.

A arquitetura utilizada, conforme (OPOCHINSKY et al., 2020), possui 5 camadas com 1024, 256, 10, 256 e 1024 neurônios, do encoder para o decoder. Em cada camada foi utilizada a normalização de lote e a função de ativação ELU (Unidade Linear Exponencial), que possui características lineares e exponenciais que oferece vantagens sobre funções tradicionais, como a ReLU, principalemente em quando ocorre o treinamento utilizando normalização de lote e permite o aprendizado mais robusto e estável de representações (DJENOURI et al., 2023). O gráfico da ELU está apresentada na figura 4.Na última camada do decoder foi utilizada a Sigmoid. O tamanho de mini-lotes foi definido para 256 e o padrão métrico para avaliação de desempenho dos modelos foi o adjusted Rand index (ARI). Foram definidas 50 épocas para o pré-treinamento e o treinamento. Também para as duas fases de treinamento foi usado o otimizador ADAM, com taxa de aprendizado 10^{-3}, e a função de perda foi a entropia cruzada binária. Essas escolhas foram baseadas nas informações apresentadas em (OPOCHINSKY et al., 2020). Os dados de entrada foram normalizados para ficarem com valores entre 0 e 1.

Embora o artigo use o TensorFlow, este projeto utilizou a biblioteca PyTorch, que apresenta funcionalidades equivalentes. Outra diferença relevante é que não foram utiliza-

Figura 3 – Pseudocódigo do algoritmo k-DAE.

Objetivo: Agrupar $x_1, \ldots, x_n \in \mathbb{R}^d$ em k grupos.

Componentes da rede:

- Conjunto de autoencoders (uma para cada grupo):

$$x \to \hat{x}(i) = f_i(x; \theta_i), \qquad i = 1, \ldots, k$$

Pré-treinamento:

- Treinar um único autoencoder com toda a base de dados.
- Aplicar o algoritmo k-means no espaço latente.
- Usar o agrupamento feito pelo k-means para inicializar os parâmetros da rede.

Treinamento: Agrupamento é obtido pela minimização do erro de reconstrução:

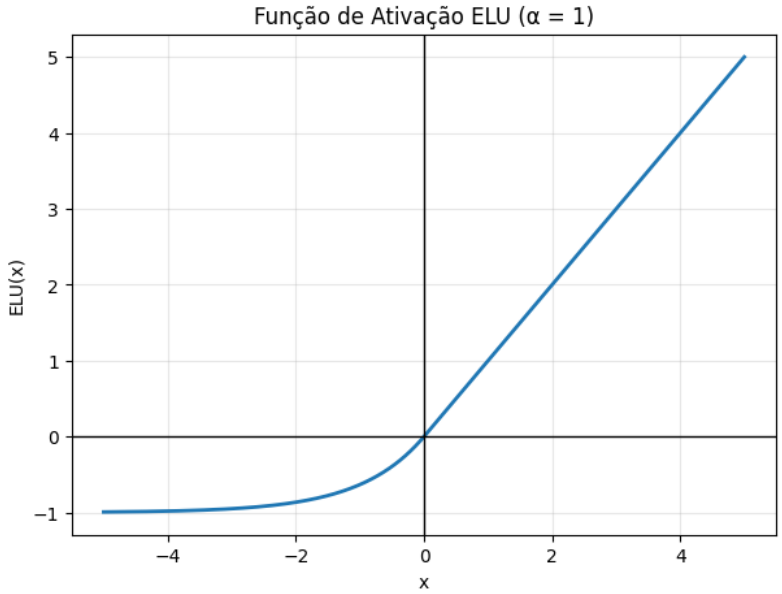
$$L(\theta_1, \ldots, \theta_k) = \sum_{t=1}^{n} \min_i d(x_t, \hat{x}_t(i))$$

Agrupamento (rígido) final:

$$\hat{c}_t = \arg\min_{i=1,\ldots,k} d(x_t, \hat{x}_t(i)), \qquad t = 1, \ldots, n.$$

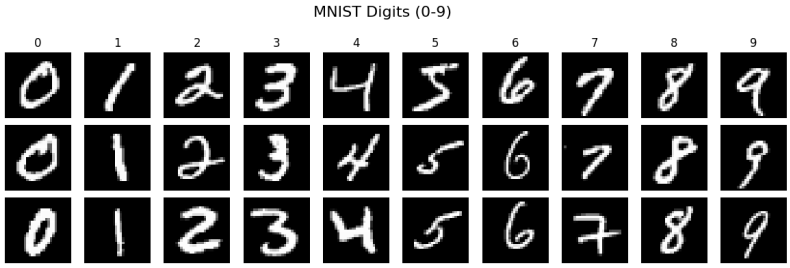
Fonte: adaptação feita pelo autor a partir do pseudocódigo apresentado em (OPOCHINSKY et al., 2020).

Figura 4 – Representação da função de ativação ELU para o valor padrão definido por α



Fonte: autor.

Figura 5 – Os rótulos do MNIST e suas representações.



Fonte: autor.

das a Parada Antecipada, nem o treinamento guloso por camada. Essas técnicas, presentes em (OPOCHINSKY et al., 2020), têm impacto direto no desempenho dos modelos e visam prevenir má convergência e melhorar a performance (BENGIO et al., 2006). Dessa forma, os resultados obtidos em (OPOCHINSKY et al., 2020) não foram reproduzidos neste trabalho de conclusão de curso.

Após a implementação da arquitetura e do processo de treinamento descritos anteriormente, todos os modelos foram avaliados seguindo um mesmo fluxo experimental. Cada modelo, Autoencoder Padrão (AE), DAE e SAE, foi treinado utilizando o fluxo do k-DAE: pré-treinamento inicial do autoencoder, aplicação do k-means no espaço latente, pré-treinamento especializado por grupo e treinamento conjunto final. Esse procedimento foi comum a todos a todos os experimentos.

3.1 Base de dados

A base de dados escolhida para os experimentos foi o MNIST (Modified National Institute of Standards and Technology), que também é utilizada no artigo (OPOCHINSKY et al., 2020). É um dos mais famosos conjuntos de dados utilizados em aprendizado de máquina, funcionando como "porta de entrada"para iniciar estudos na área e também como um padrão de comparação em pesquisas. Possui 70 mil imagens de dígitos manuscritos em escala de cinza, sendo 60 mil para treinamento e 10 mil para teste, com formato 28x28 pixels, resultando em 784 pixels por imagem. Os dígitos vão de 0 à 9, sendo cada dígito um rótulo (figura 5).

3.1.1 Experimento 1

Neste experimento, avaliou-se o desempenho dos modelos de autoencoder AE, DAE e SAE, utilizando a métrica ARI. O objetivo foi investigar como a dimensão do espaço latente (L) impacta a capacidade de agrupamento dos modelos. As dimensões avaliadas foram $L \in \{2, 10, 18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98\}$. Esses valores foram escolhidos desta forma para ter a dimensão 10 de referência e foi decidido um passo 8 entre as dimensões para ter um valor próximo à 0 e o obter uma quantidade de variações satisfatórias, considerando um intervalo de 0 à 100. No modelo DAE, o ruído foi introduzido através de

dropout na camada de entrada, com taxa de 50%. E para o SAE, a taxa de regularização λ foi definida em 10^{-4}.

3.1.2 Experimento 2

Neste experimento, foi avaliado exclusivamente o modelo DAE, variando a porcentagem de dropout aplicada na camada de entrada. A taxa de dropout variou de 0.1 a 0.9, ao passo de 0.1, com o objetivo de analisar o impacto do ruído na capacidade de reconstrução e agrupamento do modelo.

3.1.3 Experimento 3

O objetivo deste experimento foi avaliar o efeito da penalização de esparsidade no desempenho do SAE variando o λ, para os valores $\{10^0, 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$. Para observar como diferentes níveis de esparsidade influenciam a qualidade das representações latentes e o desempenho do agrupamento profundo.

Capítulo 4

Resultados e Discussões

4.1 Resultados

4.1.1 Experimento 1

Os resultados apresentados na figura 6 indicam que todos os modelos produziram espaços latentes relevantes para o agrupamento. Tanto o autoencoder padrão (AE) quanto o esparso (SAE) alcançaram melhor desempenho quando a dimensão do espaço latente foi definida igual ao número de classes do MNIST (10 dígitos), gerando representações adequadas para o k-means.

Ao aplicar o k-DAE (figura 7), observa-se que os modelos AE e SAE mantiveram desempenho satisfatório com dimensão latente igual ao número de dígitos. No entanto, ambos apresentaram queda de desempenho para dimensões maiores, indicando sensibilidade à escolha do tamanho do espaço latente. O DAE, por sua vez, apresentou resultados consistentes ao longo de todas as dimensões avaliadas, mostrando estabilidade e robustez na generalização.

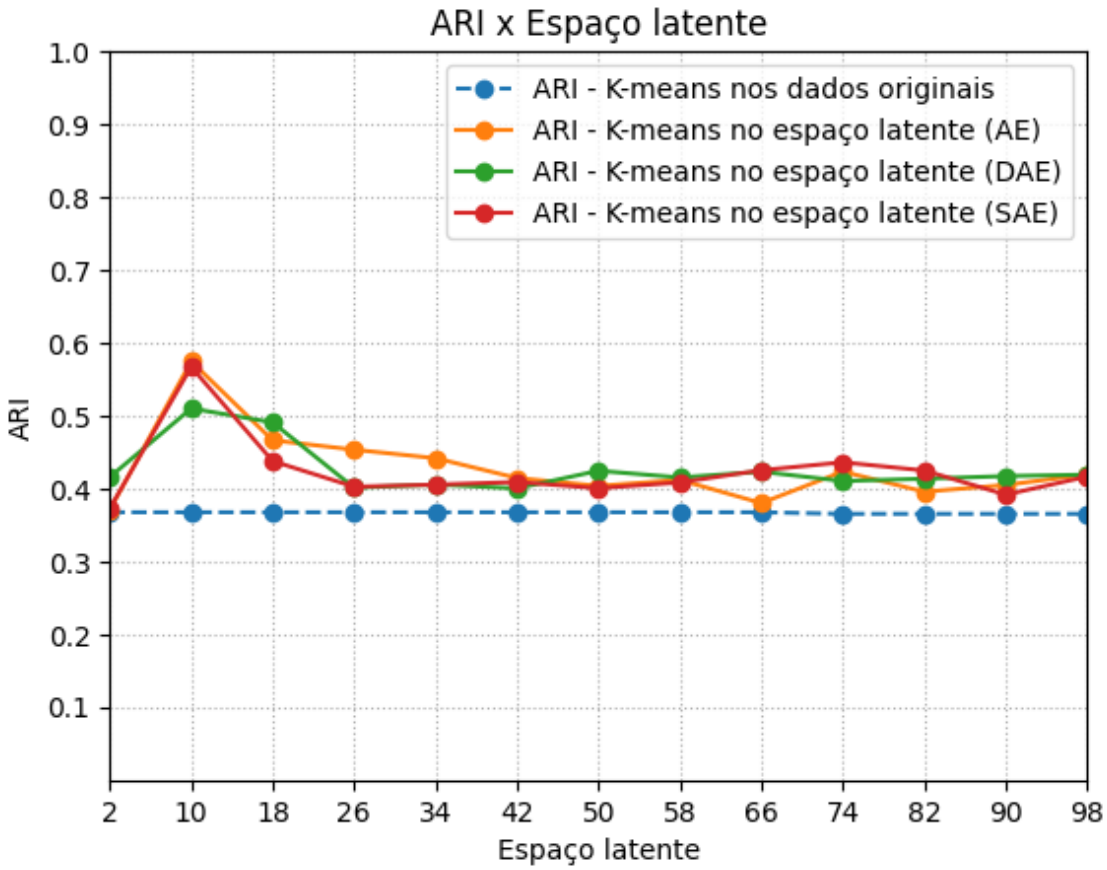
Quando comparado ao k-means aplicado sobre o espaço latente do autoencoder pré-treinado, o k-Denoising-DAE destacou-se como o único modelo capaz de superar consistentemente o desempenho do k-means em todas as dimensões do espaço latente. Os outros modelos que utilizaram o algoritmo k-DAE (AE e SAE) só apresentaram desempenho superior ao k-means quando a dimensão latente era igual a 10.

4.1.2 Experimento 2

A influência da variação do dropout na capacidade de reconstrução do modelo DAE é mostrada na figura 8. Observa-se que o desempenho é relativamente estável, mas apresenta uma tendência de queda no desempenho à medida que a taxa de neurônios desligados aumenta.

O k-Denoising-DAE apresentou desempenho superior ao k-means em todas as taxas de dropout, reforçando que a adição de ruído controlado durante o treinamento melhora a capacidade de desempenho de um modelo no agrupamento profundo.

Figura 6 – Comparativo de desempenho (rótulos produzidos pelo k-means sobre o espaço latente gerado pelo autoencoder inicial pré-treinado de cada modelo) pela variação tamanho do espaço latente.



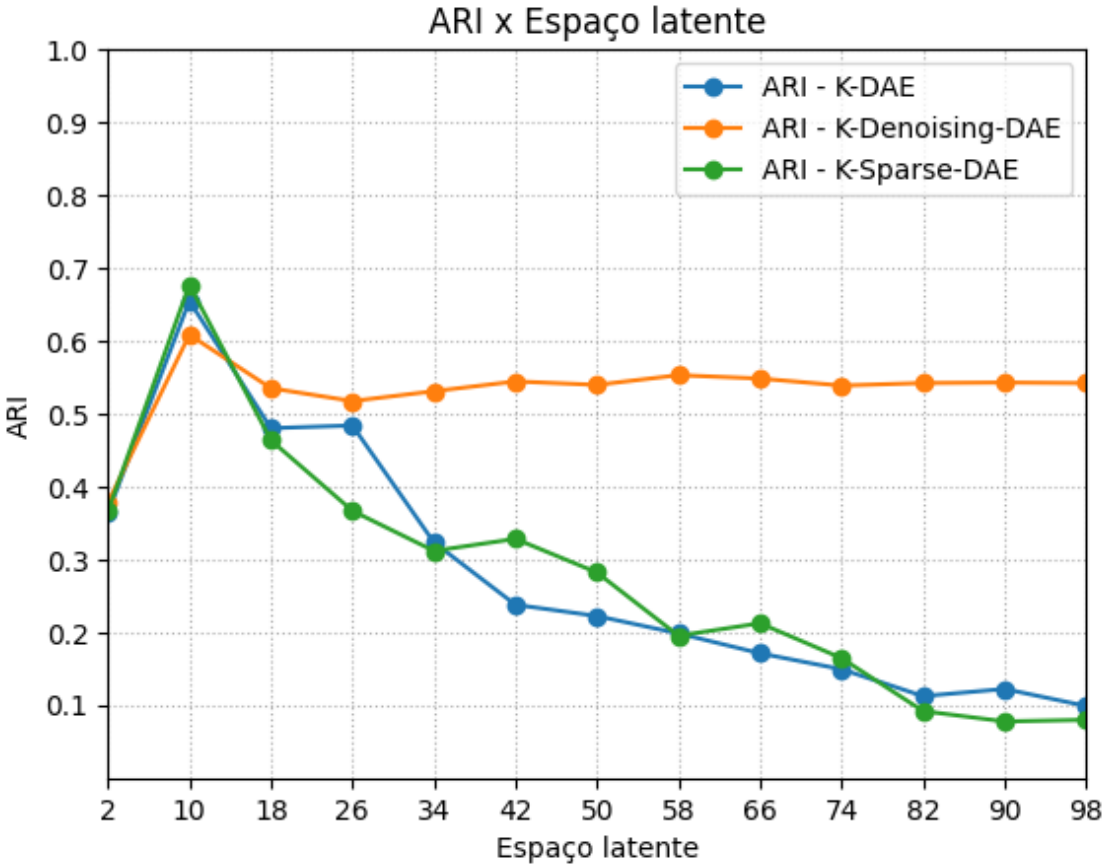
Fonte: autor.

4.1.3 Experimento 3

A figura 9 apresenta o impacto da variação do hiperparâmetro de regularização λ no desempenho do k-Sparse-DAE. Observa-se que valores maiores de λ tendem a reduzir o desempenho, indicando que uma penalização excessiva compromete a qualidade das representações latentes e do agrupamento.

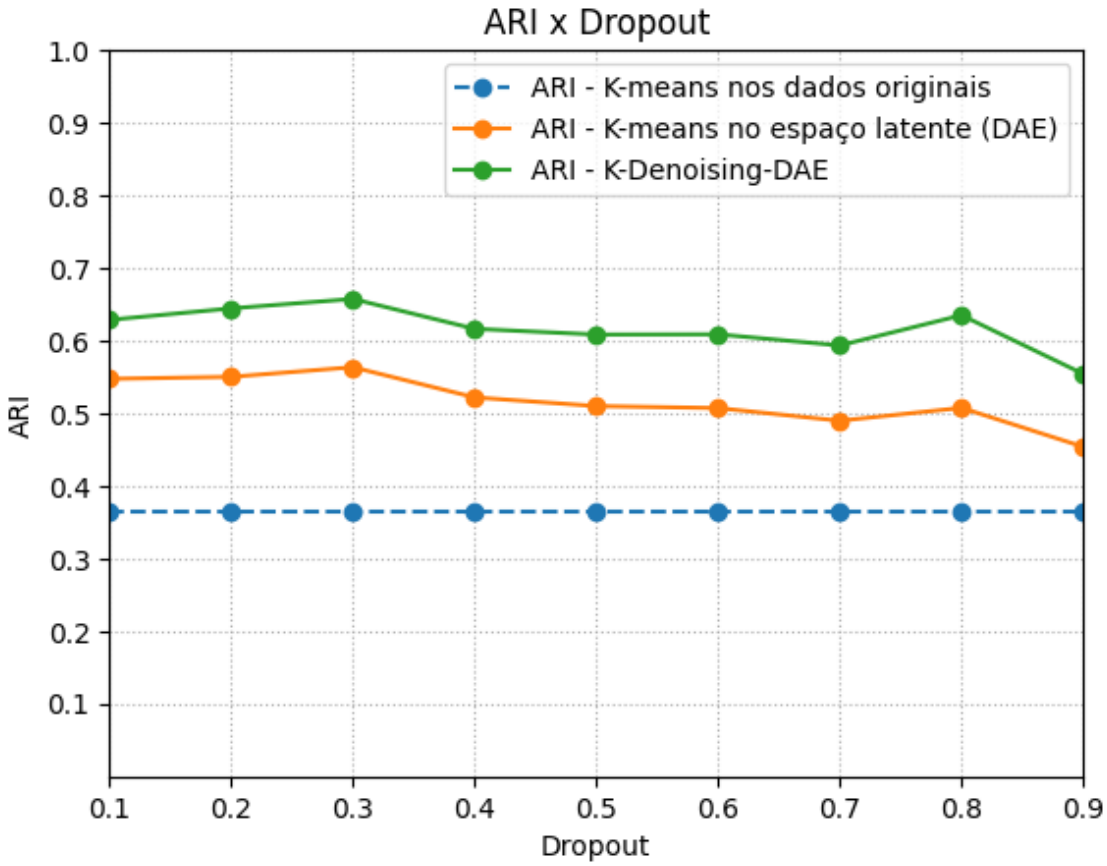
O valor de $\lambda = 10^{-4}$ utilizado neste estudo mostrou-se adequado, por ter sido o valor com o melhor desempenho dentro das opções avaliadas, equilibrando esparsidade e capacidade de reconstrução.

Figura 7 – Comparativo de desempenho (rótulos produzidos pelos K autoencoders) pela variação tamanho do espaço latente.



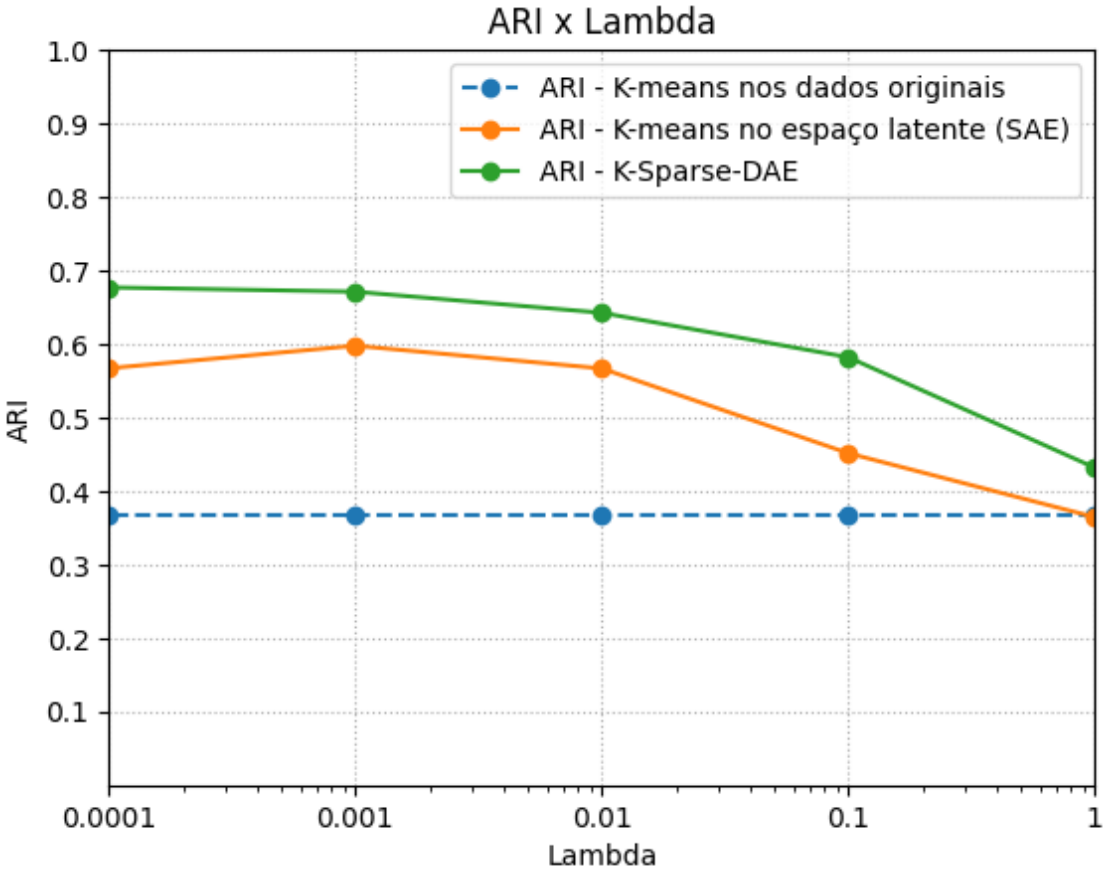
Fonte: autor.

Figura 8 – Comparativo de desempenho (rótulos produzidos pelos K DAE) pela variação de dropout.



Fonte: autor.

Figura 9 – Comparativo de desempenho (rótulos produzidos pelos K SAE) pela variação do λ.



Fonte: autor.

Capítulo 5

Conclusão

A partir dos resultados apresentados fica clara a robustez em relação à variação do espaço latente e do dropout do modelo DAE ao ser utilizado na abordagem do k-DAE, mantendo a estabilidade diante da variação da dimensionalidade do gargalo e da adição de ruído. Esse comportamento do denoising corresponde ao esperado de modelos que incorporam restrições adicionais para melhorar desempenho e generalização, como mostrado nos experimentos 1 e 2, e na literatura.

Entretanto, a restrição inserida pela penalização no SAE não garantiu uma melhoria significativa em relação ao desempenho do AE e também não se tornou robusto para generalização. Isso indica que a esparsidade imposta pelo termo de regularização λ não foi suficiente para produzir representações latentes mais discriminativas.

Para a transparência deste estudo, é importante destacar as limitações metodológicas que impactaram o desempenho dos modelos. A não utilização da Parada Antecipada (Early Stopping) e do treinamento guloso por camada (greedy layer-wise training), como mencionado na metodologia, representa um desvio significativo em relação ao procedimento de otimização adotado no artigo de referência (OPOCHINSKY et al., 2020). Essas técnicas são essenciais para garantir a estabilidade do treinamento e a otimização das representações em Redes Neurais Profundas (BENGIO et al., 2006), e a hipótese levantada é de que a sua ausência seja a causa mais provável para o desempenho geral dos modelos ter sido inferior ao k-DAE reportado na literatura.

Ainda assim, os resultados confirmam que o k-DAE, ao utilizar o número de autoencoders K igual ao número de rótulos, supera o k-means tradicional, produzindo agrupamentos mais coerentes e representativos.

Referências

BENGIO, Y. et al. Greedy layer-wise training of deep networks. In: *Advances in Neural Information Processing Systems (NeurIPS) 19*. [s.n.], 2006. Disponível em: <https://papers.nips.cc/paper/3048-greedy-layer-wise-training-of-deep-networks.pdf>. 27, 35

BISHOP, C. M.; BISHOP, H. *Deep Learning: Foundations and Concepts*. Cham: Springer, 2024. 17, 19, 20, 21

CHAZAN, S.; GANNOT, S.; GOLDBERGER, J. Deep clustering based on a mixture of autoencoder. In: *IEEE Machine Learning for Signal Processing Workshop (MLSP)*. S.l.: [s.n.], 2019. Acesso em: 16 nov. 2025. Disponível em: <https://www.eng.biu.ac.il/goldbej/files/2019/08/MLSP_Shlomi.pdf>. 14

DJENOURI, Y. et al. Fast and accurate deep learning framework for secure fault diagnosis in the industrial internet of things. *IEEE Internet of Things Journal*, v. 10, n. 4, p. 2802–2810, 2023. 25

FACELI, K. et al. *Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina*. S.l.: LTC, 2011. 13, 19

GéRON, A. *Hands-On Machine Learning with Scikit-Learn: Concepts, Tools and Techniques to Build Intelligent Systems, Keras & tensorFlow*. [S.l.]: O'Reilly, 2023. 20

LUGER, G. F. *Inteligência Artificial*. 6. ed. S.l.: Pearson, 2013. 13, 14

MAINI, V.; SABRI, S. *Machine Learning for Humans*. S.l.: Self-published, 2017. 17

MURPHY, K. P. *Probabilistic Machine Learning: An Introduction*. Cambridge, Massachusetts: The MIT Press, 2022. (Adaptive Computation and Machine Learning Series). 13, 14, 17, 18

OPOCHINSKY, Y. et al. K-autoencoders deep clustering. In: *ICASSP 2020 – IEEE International Conference on Acoustics, Speech and Signal Processing*. S.l.: [s.n.], 2020. Acesso em: 16 nov. 2025. Disponível em: <https://www.eng.biu.ac.il/goldbej/files/2020/02/ICASSP_2020_Yaniv.pdf>. 14, 21, 22, 23, 25, 26, 27, 35

PRINCE, S. J. D. *Understanding Deep Learning*. The MIT Press, 2023. Disponível em: <http://udlbook.com>. 13, 14

XU, R.; Wunsch II, D. C. *Clustering*. [S.l.]: IEEE-Wiley, 2009. 18